

Metamagic Rods — Detailed System Specification

Companion to: rods_implementation_plan.md
Scope: Complete design for all 18 metamagic rod variants
Source: DMG 3.5e pp. 235–236, SRD
Date: May 2026

Overview

Metamagic rods allow a spellcaster to apply a metamagic feat to a spell **without increasing the spell slot level**. This is one of the most impactful item categories for spellcasters in the game.

Key Rules (RAW)

- 1. **Uses per day:** 3 for all metamagic rods
- 2. **Activation:** Use-activated — wielder decides to apply when casting
- 3. **One rod per spell:** Cannot use two metamagic rods on the same spell
- 4. **Can combine with feats:** Rod + wielder’s own metamagic feat is legal; only the feat raises the slot level
- 5. **Does NOT grant the feat:** Possessing a rod does not give the wielder the metamagic feat
- 6. **Sorcerer restriction:** Spontaneous casters still use a full-round action (same as applying metamagic they know)
- 7. **Must be wielded:** Rod must be held in hand when casting the spell
- 8. **All rods:** CL 17th, Strong (no school)

Complete Rod Catalog

Price Table

Metamagic	Feat Level Adj.	Lesser (≤ 3 rd)	Normal (≤ 6 th)	Greater (≤ 9 th)
Enlarge Spell	+1	3,000 gp	11,000 gp	24,500 gp
Extend Spell	+1	3,000 gp	11,000 gp	24,500 gp
Silent Spell	+1	3,000 gp	11,000 gp	24,500 gp
Empower Spell	+2	9,000 gp	32,500 gp	73,000 gp
Maximize Spell	+3	14,000 gp	54,000 gp	121,500 gp
Quicken Spell	+4	35,000 gp	75,500 gp	170,000 gp

Tier Spell Level Limits

Tier	Max Spell Level	Example
Lesser	3rd	Fireball (3rd), Magic Missile (1st), Scorching Ray (2nd)
Normal	6th	Disintegrate (6th), Chain Lightning (6th), Greater Dispel (6th)
Greater	9th	Wish (9th), Meteor Swarm (9th), Time Stop (9th)

Metamagic Effect Definitions

1. Empower Spell

Level Adjustment: +2

Effect: All variable, numeric effects of an empowered spell are increased by one-half.

Implementation:

```
public static int ApplyEmpower(int rolledValue)
{
    // Add 50% of rolled value (round down)
    return rolledValue + (rolledValue / 2);
}

// Example: Fireball rolls 8d6 = 28 damage
// Empowered: 28 + 14 = 42 damage
// Note: Saving throws and non-variable effects are NOT affected
```

What it affects:

- Damage dice (Fireball, Lightning Bolt, etc.)
- Healing dice (Cure spells)
- Duration dice (if variable, e.g., 1d4+1 rounds)
- Number of targets (if variable)

What it does NOT affect:

- Fixed bonuses (+1 per caster level portions)
- Save DCs
- Caster level checks
- Non-numeric effects

2. Enlarge Spell

Level Adjustment: +1

Effect: Double the range of the spell.

Implementation:

```

public static int ApplyEnlarge(int baseRange)
{
    return baseRange * 2;
}

// Touch spells become Close range (25 ft + 5 ft/2 levels)
// Close becomes Medium (doubled)
// Medium becomes Long (doubled)
// Does NOT affect spells with range "Personal"

```

Rules:

- Touch range → Close range (25 ft + 5 ft/2 levels)
- Close/Medium/Long ranges → doubled
- Personal range → NOT affected
- Fixed-range spells (e.g., 60 ft cone) → doubled

3. Extend Spell

Level Adjustment: +1**Effect:** Double the duration of the spell.**Implementation:**

```

public static int ApplyExtend(int baseDurationRounds)
{
    return baseDurationRounds * 2;
}

// Example: Haste duration 1 round/level at CL 10 = 10 rounds
// Extended: 20 rounds
// Does NOT affect instantaneous or permanent durations

```

Rules:

- Multiply duration by 2
- Instantaneous duration → NOT affected
- Permanent duration → NOT affected
- Concentration duration → NOT affected (concentration is not a fixed duration)

4. Maximize Spell

Level Adjustment: +3**Effect:** All variable, numeric effects are maximized (treated as maximum possible roll).**Implementation:**

```

public static int ApplyMaximize(int diceCount, int dieSize)
{
    // All dice treated as maximum value
    return diceCount * dieSize;
}

// Example: Fireball 10d6
// Normal: average 35 (range 10-60)
// Maximized: 60 (all 6s)

```

What it affects: Same categories as Empower

Interaction with Empower: Can stack (via feat + rod combo):

- Maximized + Empowered Fireball $10d6 = 60 + 30 = 90$ damage

5. Quicken Spell

Level Adjustment: +4

Effect: Cast the spell as a swift action instead of standard action.

Implementation:

```
public static CastingTime ApplyQuicken(CastingTime baseCastingTime)
{
    // Only works on spells with casting time of 1 standard action
    if (baseCastingTime == CastingTime.StandardAction)
        return CastingTime.SwiftAction;

    return baseCastingTime; // Cannot quicken longer casting times
}
```

Rules:

- Only works on spells with casting time of 1 standard action
- Cannot quicken spells with longer casting times
- Quickened spell does NOT provoke attacks of opportunity
- Only ONE quickened spell per turn (swift action limit)
- Caster can still cast a normal spell the same turn (standard action)

6. Silent Spell

Level Adjustment: +1

Effect: Cast the spell without verbal components.

Implementation:

```
public static bool ApplySilent(SpellComponents components)
{
    // Remove verbal component requirement
    components.Verbal = false;
    return true;
}

// Useful when:
// - Silenced (Silence spell)
// - Gagged/bound
// - Underwater
// - Stealth casting
```

Rules:

- Removes verbal (V) component only
 - Somatic (S) and Material (M) components still required
 - Spell with no verbal component cannot benefit (redundant)
-

System Architecture

Core Classes

```

/// <summary>
/// Enum for all metamagic types supported by rods.
/// </summary>
public enum MetamagicType
{
    Empower,
    Enlarge,
    Extend,
    Maximize,
    Quicken,
    Silent
}

/// <summary>
/// Rod power tier determining max spell level.
/// </summary>
public enum MetamagicRodTier
{
    Lesser,    // ≤ 3rd level spells
    Normal,    // ≤ 6th level spells
    Greater    // ≤ 9th level spells
}

/// <summary>
/// Data for a specific metamagic rod instance.
/// </summary>
public class MetamagicRodData
{
    public MetamagicType Type;
    public MetamagicRodTier Tier;
    public int UsesPerDay = 3;
    public int UsesRemaining = 3;

    public int MaxSpellLevel => Tier switch
    {
        MetamagicRodTier.Lesser => 3,
        MetamagicRodTier.Normal => 6,
        MetamagicRodTier.Greater => 9,
        _ => 3
    };

    public int MarketPrice => (Type, Tier) switch
    {
        (MetamagicType.Enlarge, MetamagicRodTier.Lesser) => 3000,
        (MetamagicType.Enlarge, MetamagicRodTier.Normal) => 11000,
        (MetamagicType.Enlarge, MetamagicRodTier.Greater) => 24500,
        (MetamagicType.Extend, MetamagicRodTier.Lesser) => 3000,
        (MetamagicType.Extend, MetamagicRodTier.Normal) => 11000,
        (MetamagicType.Extend, MetamagicRodTier.Greater) => 24500,
        (MetamagicType.Silent, MetamagicRodTier.Lesser) => 3000,
        (MetamagicType.Silent, MetamagicRodTier.Normal) => 11000,
        (MetamagicType.Silent, MetamagicRodTier.Greater) => 24500,
        (MetamagicType.Empower, MetamagicRodTier.Lesser) => 9000,
        (MetamagicType.Empower, MetamagicRodTier.Normal) => 32500,
        (MetamagicType.Empower, MetamagicRodTier.Greater) => 73000,
        (MetamagicType.Maximize, MetamagicRodTier.Lesser) => 14000,
        (MetamagicType.Maximize, MetamagicRodTier.Normal) => 54000,
        (MetamagicType.Maximize, MetamagicRodTier.Greater) => 121500,
        (MetamagicType.Quicken, MetamagicRodTier.Lesser) => 35000,
        (MetamagicType.Quicken, MetamagicRodTier.Normal) => 75500,
        (MetamagicType.Quicken, MetamagicRodTier.Greater) => 170000,
        _ => 0
    }
}

```

```
}    };
```

Metamagic Rod Manager


```

/// <summary>
/// Manages metamagic rod application during spell casting.
/// </summary>
public class MetamagicRodManager
{
    /// <summary>
    /// Check if character has a usable metamagic rod for a given spell.
    /// </summary>
    public static List<MetamagicRodData> GetAvailableMetamagicRods(
        Character caster, int spellLevel)
    {
        var results = new List<MetamagicRodData>();

        // Check MainHand and OffHand for metamagic rods
        foreach (var heldItem in caster.GetHeldItems())
        {
            if (heldItem.ItemType != ItemType.Rod) continue;
            if (heldItem.MetamagicData == null) continue;

            var rod = heldItem.MetamagicData;
            if (rod.UsesRemaining > 0 && spellLevel <= rod.MaxSpellLevel)
            {
                results.Add(rod);
            }
        }

        return results;
    }

    /// <summary>
    /// Apply metamagic from rod to a spell being cast.
    /// Returns modified spell parameters.
    /// </summary>
    public static SpellParameters ApplyMetamagic(
        MetamagicRodData rod, SpellParameters spell)
    {
        // Validate
        if (rod.UsesRemaining <= 0)
            throw new InvalidOperationException("Rod has no uses remaining today.");
        if (spell.Level > rod.MaxSpellLevel)
            throw new InvalidOperationException(
                $"Spell level {spell.Level} exceeds rod max {rod.MaxSpellLevel}.");

        // Consume use
        rod.UsesRemaining--;

        // Apply metamagic effect
        switch (rod.Type)
        {
            case MetamagicType.Empower:
                spell.DamageMultiplier = 1.5f; // +50%
                spell.HealingMultiplier = 1.5f;
                break;

            case MetamagicType.Enlarge:
                if (spell.Range == SpellRange.Touch)
                    spell.Range = SpellRange.Close;
                else
                    spell.RangeInFeet *= 2;
                break;

            case MetamagicType.Extend:

```

```

        spell.DurationRounds *= 2;
        break;

    case MetamagicType.Maximize:
        spell.MaximizeDice = true;
        break;

    case MetamagicType.Quicken:
        spell.CastingTime = CastingTime.SwiftAction;
        break;

    case MetamagicType.Silent:
        spell.RequiresVerbal = false;
        break;
    }

    // IMPORTANT: Spell slot is NOT changed
    // spell.SlotLevel remains the same

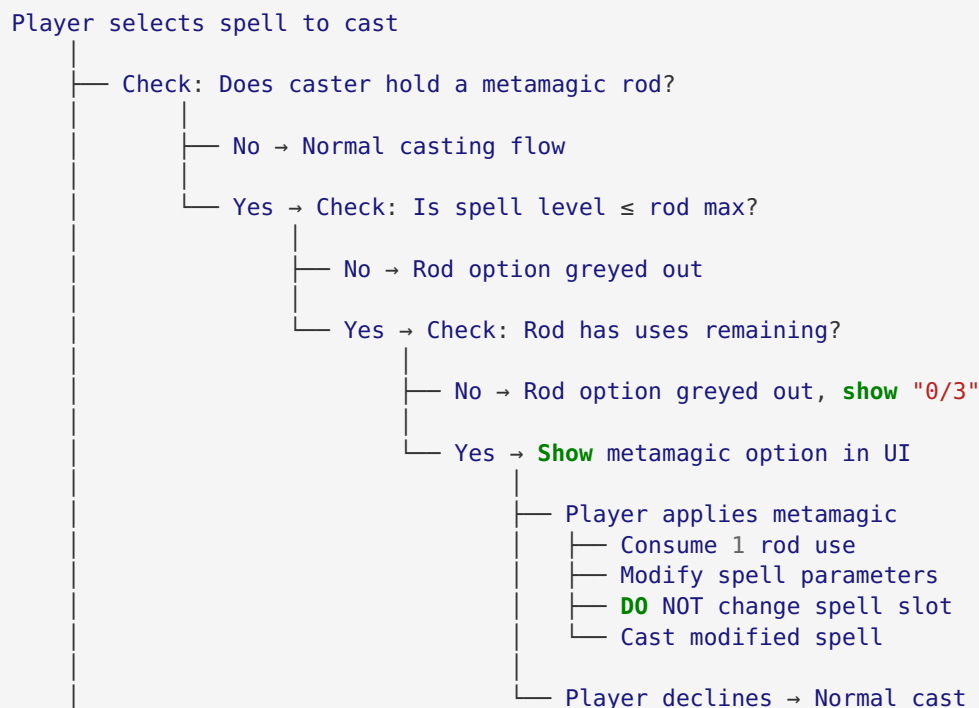
    return spell;
}

/// <summary>
/// Reset all metamagic rod uses on long rest.
/// </summary>
public static void ResetOnRest(Character caster)
{
    foreach (var item in caster.GetAllInventoryItems())
    {
        if (item.MetamagicData != null)
        {
            item.MetamagicData.UsesRemaining = item.MetamagicData.UsesPerDay;
        }
    }
}
}

```

Spell Casting UI Integration

Flow Diagram



UI Mockup

CAST SPELL: Fireball (3rd Level)

Target: [selected tile]

Damage: 10d6 fire (Reflex DC 17 half)

Range: 400 ft

Spell Slot: 3rd level

METAMAGIC ROD

☒ Empower Spell (2/3 uses left)

☒ Damage becomes 10d6 ☒ 1.5

☒ Still uses 3rd level slot

[Cast] [Cast + Empower] [Cancel]

Edge Cases & Rules Clarifications

1. Two Metamagic Rods

Rule: Only ONE metamagic rod per spell.

Implementation: Once a rod is selected, disable other rod options for this cast.

2. Rod + Own Metamagic Feat

Rule: Legal. Rod's metamagic doesn't raise slot. Feat's metamagic DOES raise slot.

Example: Wizard with Maximize feat + Rod of Empower:

- Casts Maximized Empowered Fireball
- Maximize (feat): raises slot from 3rd to 6th
- Empower (rod): does NOT raise slot
- Uses a 6th-level slot, not 8th

3. Sorcerer Full-Round Action

Rule: Spontaneous casters who apply metamagic (via feat OR rod) use full-round action.

Implementation: If caster is spontaneous AND metamagic rod is used → casting time = full-round (unless Quicken rod, which overrides to swift action).

4. Quicken + Sorcerer

Rule: Quicken rod makes spell a swift action, overriding the sorcerer full-round penalty.

Implementation: Quicken takes priority over spontaneous metamagic slowdown.

5. Rod Not Held

Rule: Rod must be actively wielded (held in hand) when the spell is cast.

Implementation: Check equipment slot at moment of casting. If rod is in inventory but not equipped → cannot use.

6. Antimagic Field

Rule: Rod doesn't function in antimagic field.

Implementation: Check for antimagic zone before offering metamagic option.

Save/Load Specification

Serialized State Per Rod

```
{
  "itemId": "rod_empower_lesser_001",
  "metamagicType": "Empower",
  "tier": "Lesser",
  "usesRemaining": 2,
  "usesPerDay": 3,
  "equippedSlot": "MainHand"
}
```

Rest Reset Logic

```
// On long rest completion
foreach (var rod in allMetamagicRods)
{
  rod.UsesRemaining = rod.UsesPerDay; // Always 3
}
```

Testing Matrix

Per Metamagic Type

Test	Empower	Enlarge	Extend	Maximize	Quicken	Silent
Save/load preserves uses	☐	☐	☐	☐	☐	☐

System-Level Tests

Test	Status
Two rods on one spell → blocked	☐
Rod + own feat → only feat raises slot	☐
Sorcerer + rod → full-round action	☐
Sorcerer + Quicken rod → swift action	☐
Rod in inventory (not held) → blocked	☐
Empower + Maximize (feat+rod combo) → correct damage	☐

Estimated Implementation Time

Component	Days
MetamagicType enum + MetamagicRodData	0.5
MetamagicRodManager core	1.5
Empower effect (damage/healing multiplier)	1
Enlarge effect (range modification)	0.5
Extend effect (duration modification)	0.5
Maximize effect (dice maximization)	0.5
Quickened effect (casting time change)	1
Silent effect (component removal)	0.5
Spell casting UI integration	2
Daily use tracking + rest reset	0.5
Save/load serialization	0.5
Rod factory (18 variants)	1
Testing all 18 variants	2
Total	~12 days